

School of Science, Computing and Engineering Technologies



COS10009

Introduction to Programming

Custom Code Report

Project Title: Super Mario Bros Clone

Student Name: Quang Huy Tran

Student ID: 106212636

Date: 12/04/2026

Declaration

I declare that this report is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person.

Signature: 

1. Project Overview

This report describes the design and development of a Super Mario Bros–inspired game created using Python and Pygame. The project focuses on recreating key elements of World 1-1 from the original 1985 Nintendo game, such as basic movement physics, enemy behaviours, collision handling, sprite animations, sound effects, and a simple game flow with multiple screens and a score system.

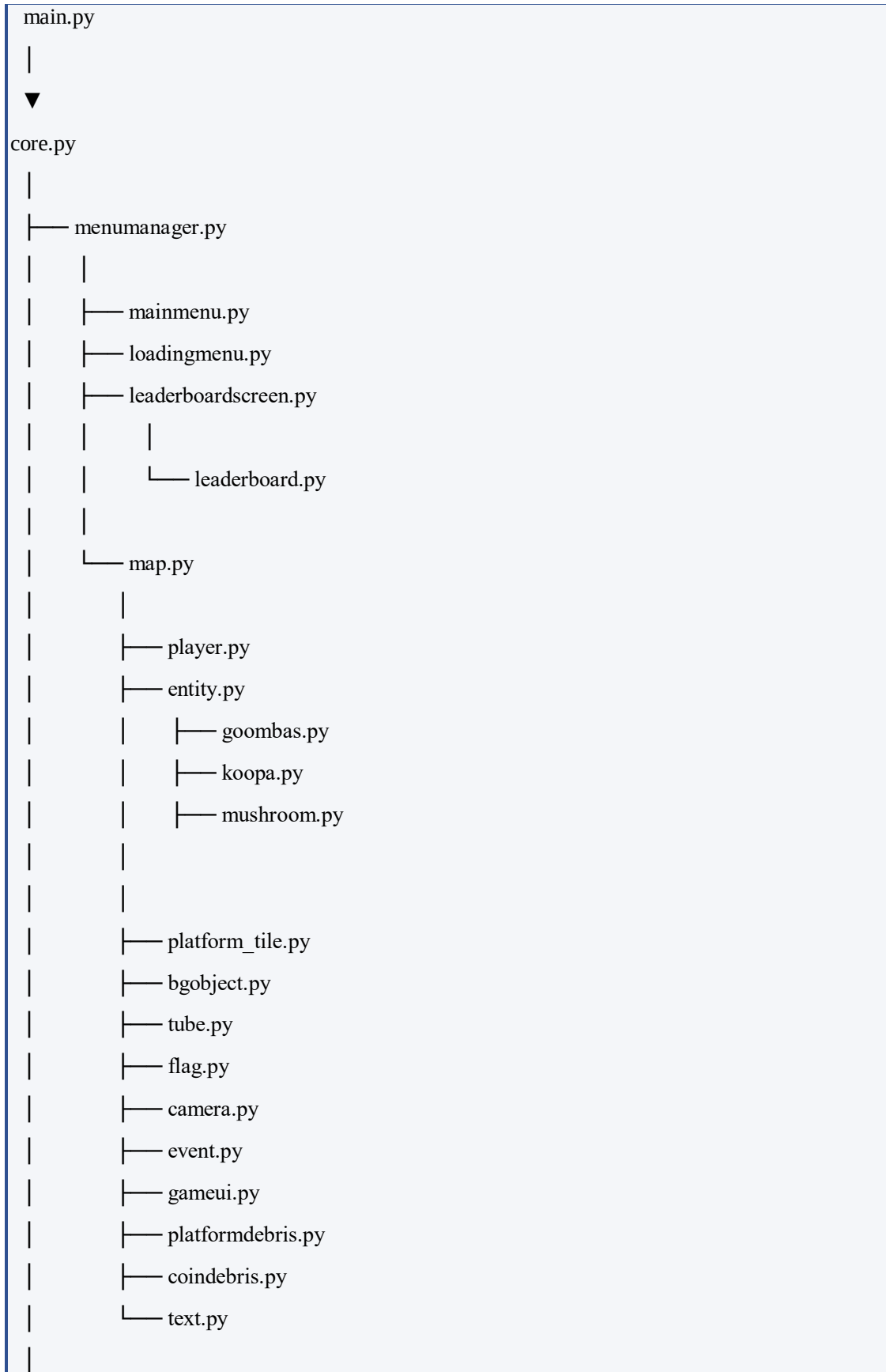
The program consists of 26 Python files. Instead of relying on a full game engine, the main features, like physics, collision detection, and animations which were implemented using core programming concepts, supported by the Pygame and pytmx libraries.

Scope: World 1-1 only. Features include: 6 pipe obstacles, 15+ enemies (Goomba and Koopa), 3 block types (question, brick, normal), a flagpole win sequence with time-bonus scoring, a 3-life death and respawn system, and a session leaderboard screen.

Property	Detail
Language	Python
Libraries	Pygame, pytmx
Source files	26.py modules
Map size	800 × 448 px
Sound effects	17 distinct audio files
Sorting algorithm	Custom Bubble Sort with early-exit optimisation

2. System Architecture

The application is divided into 26 small parts (called modules), and these are grouped into five main layers: entry point, engine, world, entities, and assets. Each module has its own simple job, so it's easier to understand and fix. Because of this, when you change one part, it usually won't affect the others or cause errors in the program.



 sound.py

3. Functional Decomposition

I break a big problem into smaller parts so each part is easier to handle. In this project, I split the program into 26 modules, and then I continue breaking each module into smaller functions that handle specific tasks.

This makes the code easier for me to work with because I can focus on one small part instead of dealing with the whole system at once. It also helps me debug and update the program more easily. For example, if there is a problem with jumping, I only need to check `player_physics()`. If the leaderboard is not sorting correctly, I just look at the sorting function instead of going through the entire code.

3.1 Module-Level Decomposition

Each module has exactly one responsibility. The most important ones are listed below:

Module	Responsibility
main.py	Entry point — creates Core and starts the game loop
core.py	Window init, main loop (input → update → render), key state booleans
menumanager.py	Game state machine — decides which screen is active each frame
const.py	All named constants for physics, window size, and FPS
map.py	World loading, 2D tile grid, all entity lists, orchestrates updates
camera.py	Side-scrolling viewport with map-boundary clamping
event.py	Scripted sequences: death bounce, flagpole win, end-of-level score tally
gameui.py	HUD rendering: score, coins, world name, time, lives
player.py	Player physics, animation state machine, power-up logic
entity.py	Abstract base class: shared physics for all moving entities
goombas.py	Goomba: walk, crush/shot death states, player collision

Module	Responsibility
koopas.py	Koopa: 4 states — walking, shell, sliding shell, dead
mushroom.py	Mushroom power-up: spawn animation, movement, player pickup
platform_tile.py	Block logic: question block animation, bonus spawning, brick break
bgobject.py	Non-collidable background decoration — render only
tube.py	Pipe with dynamic height via subsurface sprite clipping
flag.py	Flagpole: pillar and sliding flag piece
platformdebris.py	4-particle brick debris with gravity after block break
coindebris.py	Coin bounce animation from question block hit
text.py	Floating score text: drifts upward 100 px then self-removes
sound.py	Load 17 sound files, named play/stop interface
leaderboard.py	RunsTracker: add run records, sort with bubble sort, clear session
leaderboardscreen.py	Animated end-of-session results screen with 3-life ranking
loadingmenu.py	World title card with timed transition into gameplay
mainmenu.py	Title screen image and start prompt
debugtable.py	Optional debug overlay: FPS, player position, live state values

3.2 Function-Level Decomposition — player.py

I made the Player class the most complex part of the project, with 19 methods. Each method is designed to do one specific task and nothing more. A good example is the `update()` method - instead of putting all the logic in one place, I simply let it call three smaller, focused methods.

```
def update(self, core):  
    # Update the player's state every frame, including movement, animation, and invincibility time.  
    self.player_physics(core)  
    self.update_image(core)  
    self.update_unkillable_time()
```

I wrote the `update()` method this way so that each part stays separate and easier to manage. If there is a bug where the wrong animation frame shows up, I only need to

check `update_image()`. I don't have to go through the physics code to find the problem. This makes the code easier for me to debug and understand.

All 19 methods in `player.py` and their responsibilities are listed below:

Method	Responsibility
<code>__init__()</code>	Set up all starting values, load sprites, create the hitbox rectangle
<code>load_sprites()</code>	Load 50 PNG frames, mirror left-facing versions, add special frames
<code>update(core)</code>	Run per-frame: physics → image selection → unkillable timer
<code>player_physics(core)</code>	Read keyboard input, apply velocity and gravity, resolve collisions
<code>update_x_pos(blocks)</code>	Horizontal collision: snap the rect against walls, sync the float position
<code>update_y_pos(blocks, core)</code>	Vertical collision: land on blocks, head-bump, trigger block actions
<code>activate_block_action(core, block)</code>	Handle what happens when Mario bumps a question block or brick from below
<code>update_image(core)</code>	Pick the correct animation frame based on speed and ground state
<code>set_image(image_id)</code>	Calculate the sprite array index from power level and direction
<code>set_powerlvl(power_lvl, core)</code>	Handle upgrades, downgrades, and death based on current power level
<code>flag_animation_move(core, walk_to_castle)</code>	Slide Mario down the flagpole and walk him into the castle
<code>jump_on_mob()</code>	Bounce Mario upward after he stomps an enemy
<code>reset(reset_all)</code>	Reset position; if <code>reset_all</code> is True, also wipe score, lives, and coins

3.3 Function-Level Decomposition — `map.py`

I designed `map.py` to coordinate the entire game world each frame. Instead of putting all the logic into one large block of code, I split the `update()` method into smaller sub-methods, each handling a specific responsibility. This way, if something like the time counter breaks, I immediately know to check `update_time()` instead of going through a long mix of unrelated code.

```
def update(self, core):
    # Update the game state every frame.
    # This includes updating the player, mobs, projectiles, debris, and handling events like the player dying or winning.

    # All mobs
    self.update_entities(core)

    if not core.get_map().in_event:
        # When player eats a mushroom
        if self.get_player().inLevelUpAnimation:
            self.get_player().change_powerlvl_animation()

        # Unlike the level up animation, player can move there
        elif self.get_player().inLevelDownAnimation:
            self.get_player().change_powerlvl_animation()
            self.update_player(core)

        # Common case
        else:
            self.update_player(core)

    else:
        self.get_event().update(core)

    # Debris from blocks and coins
    for debris in self.debris:
        debris.update(core)

    # Player's fireballs
    for whizbang in self.projectiles:
        whizbang.update(core)

    # Text which represent how many points player get
    for text_object in self.text_objects:
        text_object.update(core)

    # Camera stops moving when player dies or touches a flag
    if not self.in_event:
        self.get_camera().update(core.get_map().get_player().rect)
```

I added an `in_event` check at the top because when Mario dies or reaches the flagpole, a scripted cutscene takes over. During this time, I don't want normal player input to run, so I skip `update_player(core)` and run `get_event().update(core)` instead. This is an example of using selection (if/else) to switch between two completely different behaviours depending on the current game state.

4. Arrays and Record

4.1 Arrays (Python Lists)

I use Python lists throughout the project to store groups of game objects that need to be updated and drawn every frame. I choose lists instead of fixed-size arrays because the number of objects is not constant - enemies can spawn or die, fireballs can be created and removed, and brick debris can appear and disappear during gameplay. Most of the important lists are stored in `map.py`, and I manage them centrally so other modules can access them through `core`.

List	Stores	How it changes during gameplay
------	--------	--------------------------------

self.obj	Platform tiles	Tiles are removed from this list when a brick block is broken
self.mobs	Enemy objects	Enemies are added on spawn and removed when they die or fall off screen
self.debris	Debris particles	Created when a brick breaks; each particle removes itself when it lands
self.projectiles	Fireball objects	Capped at 2 at once; each fireball removes itself on impact
self.text_objects	Floating score text	Created on kills/pickups; each one removes itself after drifting 100 px up
self.sprites (player.py)	50 sprite surfaces	Loaded once at start; never changes during gameplay
self.runs (leaderboard.py)	Run records (dicts)	One record is added per life played, up to 3 per session

To understand how list removal works, here is the pattern used by debris particles. Each particle calls `core.get_map().debris.remove(self)` on itself when it finishes - this is a clean way to let objects clean themselves up without needing a separate manager:

```

self.y_vel = -4
self.rect = None

def update(self, core):
    # Apply gravity so the pieces fall down over time
    self.y_vel += GRAVITY * FALL_MULTIPLIER

    # Loop through each of the 4 pieces to move them
    for i in range(len(self.rectangles)):
        # Update the vertical position based on velocity
        self.rectangles[i].y += self.y_vel
        # Make the first two pieces move left and the other two move right
        if i < 2:
            self.rectangles[i].x -= 1
        else:
            self.rectangles[i].x += 1

    # If the debris falls below the bottom of the map, delete it to keep the game fast
    if self.rectangles[1].y > core.get_map().mapSize[1] * 32:
        core.get_map().debris.remove(self)

```

Without the `remove()` call, the list would keep growing every time a brick breaks and never shrink. Over a long play session this would waste memory and slow down the game.

4.2 Records — Classes as Structured Data

In Python, I use classes like a “record” to group related data together. Instead of keeping separate variables for a player’s score, lives, velocity, and position, I put everything inside the Player class.

The Player class keeps all of Mario’s state in one place. This makes it easier for me to work with the player throughout the program, since I can pass around a single object instead of managing many individual variables.

```
class Player(object):
    def __init__(self, x_pos, y_pos):
        # Player parameters
        self.numOfLives = 3
        self.score = 0
        self.coins = 0

        self.visible = True
        self.spriteTick = 0
        self.powerLVL = 0

        self.unkillable = False
        self.unkillableTime = 0

        self.inLevelUpAnimation = False
        self.inLevelUpAnimationTime = 0
        self.inLevelDownAnimation = False
        self.inLevelDownAnimationTime = 0

        self.already_jumped = False
        self.next_jump_time = 0
        self.next_fireball_time = 0
        self.x_vel = 0
        self.y_vel = 0
        self.direction = True
        self.on_ground = False
        self.fast_moving = False

        self.pos_x = x_pos
```

I separate `pos_x` and `rect.x` because they serve different purposes. I keep `pos_x` as a float so it can store smooth movement with small decimal values every frame, while `rect.x` is an integer because pixels on the screen can’t be fractional and it’s what I use for collisions and drawing.

If I only used `rect.x`, the values would keep getting rounded every frame. Over time, this small loss of precision can build up and make movement feel off, like the player slowing down or getting stuck at certain speeds.

5. Structured Programming

In this project, I rely on three main control flow patterns throughout the code: doing steps in order, making decisions with conditions, and repeating actions using loops. The examples below show how I apply each of these and why they are needed in different parts of the game.

5.1 Sequence

In my programme, sequence can be used run instructions in a specific order, and if I change that order, the program will break. A clear example is my main game loop in `core.py`, where I always run three steps including input, update, and render in the right order

```
def main_loop(self):
    # The main game loop: get input, update physics, and draw everything
    while self.run:
        self.input()
        self.update()
        self.render()
        self.clock.tick(FPS) # Keep the game running at a steady speed
```

If I ran `render()` before `update()`, the player would see a frame that is always one step behind, making the game feel like it is lagging even on a fast machine. If I ran `input()` after `update()`, then the player's key press would only affect the next frame, which would make the controls feel delayed and unresponsive.

5.2 Selection

I use selection throughout my project with `if / elif / else` statements to control what happens in different situations. For example, I decide which screen to show based on the current game state, and I also adjust Mario's falling speed depending on what condition he is in during gameplay.

Choosing input mode:

The first decision my game makes each frame is which input mode to use. During gameplay, I handle arrow keys for movement, as well as jump, run, and shoot actions. However, when the player is on any menu screen, I only process the Enter key, and ignore all other inputs.

```
def input(self):
    # Choose which input logic to use based on the current screen
    if self.get_mm().currentGameState == 'Game':
        self.input_player()
    else:
        self.input_menu()
```

Without this check, pressing an arrow key on the main menu would try to control a player object that does not exist yet, which would cause the program to crash.

Variable-height jump physics:

The jump in my game works differently depending on whether I hold or release the jump button. This is a “variable jump” technique, similar to the original Mario game. I implement it using three mutually exclusive conditions:

```
if 0 < self.x_vel < SPEED_DECREASE_RATE:  
    self.x_vel = 0  
if 0 > self.x_vel > -SPEED_DECREASE_RATE:  
    self.x_vel = 0  
  
if not self.on_ground:  
    # Moving up, button is pressed  
    if (self.y_vel < 0 and core.keyU):  
        self.y_vel += GRAVITY  
  
    # Moving up, button is not pressed - low jump  
    elif (self.y_vel < 0 and not core.keyU):  
        self.y_vel += GRAVITY * LOW_JUMP_MULTIPLIER  
  
    # Moving down  
    else:  
        self.y_vel += GRAVITY * FALL_MULTIPLIER  
  
    if self.y_vel > MAX_FALL_SPEED:  
        self.y_vel = MAX_FALL_SPEED
```

LOW_JUMP_MULTIPLIER (3.0) is what I use when the player releases the jump button early; it increases gravity so the upward movement stops quickly. FALL_MULTIPLIER (2.0) makes Mario fall faster than he rises, which gives the jump a more responsive, “snappy” feeling. I also cap terminal velocity so he never falls too fast and clips through thin platforms.

5.3 Repetition

Entity iteration - for:

Every frame, my game has to update all enemies and draw all tiles. I use a for loop here because I need to apply the same operation — either updating or rendering — to every item in a list, one by one.

```
def update_entities(self, core):  
    # Update all mobs and check for collisions with the player.  
    for mob in self.mobs:  
        mob.update(core)  
        if not self.in_event:  
            self.entity_collisions(core)
```

```
for obj_group in (self.obj_bg, self.obj):  
    for obj in obj_group:  
        obj.render(core)  
  
for tube in self.tubes:  
    tube.render(core)
```

```
for debris in self.debris:  
    debris.render(core)
```

Nested loops - loading the tile map:

When my game starts, I read every tile from the TMX map file and create either a Platform object or a BGObject for it. Since the map is 800x448, I use a nested loop with two range() calls to go through every (x, y) position and build the level tile by tile.

```
66     for layer in tmx_data.visible_layers:
67         # Loop through every tile in the layer and create the appropriate game objects based on the tile ID and layer name.
68         for y in range(tmx_data.height):
69             for x in range(tmx_data.width):
70
71                 # Getting pygame surface
72                 image = tmx_data.get_tile_image(x, y, layer_num)
73
74                 # It's none if there are no tile in that place
75                 if image is not None:
76                     tileID = tmx_data.get_tile_gid(x, y, layer_num)
77
78                 if layer.name == 'Foreground':
79
80                     # Special case for "question" blocks: they have 4 frames (1 normal, 3 activated) and I need to load them all so
81                     if tileID == 22:
82                         image = (
83                             image, # 1
84                             tmx_data.get_tile_image(0, 15, layer_num), # 2
85                             tmx_data.get_tile_image(1, 15, layer_num), # 3
86                             tmx_data.get_tile_image(2, 15, layer_num) # activated
87                         )
88
89                     # For all other tiles, I just create a Platform object with the image and tile ID. I also add it to the list of ob
90                     # The map grid (self.map) is a 2D list where each cell can either be 0 (empty) or contain a Platform or BGObject i
91                     # This allows for easy collision detection later on when the player or mobs interact with the environment.
92                     self.map[x][y] = Platform(x * tmx_data.tileheight, y * tmx_data.tilewidth, image, tileID)
93                     self.obj.append(self.map[x][y])
94
95                 elif layer.name == 'Background':
96                     self.map[x][y] = BGObject(x * tmx_data.tileheight, y * tmx_data.tilewidth, image)
97                     self.obj_bg.append(self.map[x][y])
98             layer_num += 1
99
```

The $x * 32$ and $y * 32$ calculations convert tile coordinates into pixel coordinates, since each tile is 32×32 pixels in size. I store all tiles in a 2D `self.map` list using their grid positions, so I can easily access them later. This structure is also used by my spatial collision system to quickly find nearby tiles instead of checking every tile in the entire map.

6. Algorithm — Bubble Sort

```
def get_sorted_runs(self):
    # Copy the list so the original run order is not changed.
    runs = self.runs[:]
    n = len(runs)

    # We move the worst ranked run toward the end of the list repeatedly.
    for i in range(n):
        swapped = False
        for j in range(0, n - 1 - i):
            left = runs[j]
            right = runs[j + 1]

            # Compare by score first: higher score wins.
            # If scores are equal, compare by time: lower time wins.
            if left['score'] < right['score']:
                should_swap = True
            elif left['score'] > right['score']:
                should_swap = False
            else:
                should_swap = left['time'] > right['time']

            if should_swap:
                runs[j], runs[j + 1] = runs[j + 1], runs[j]
                swapped = True

        # If no swaps happened, the list is already sorted.
        if not swapped:
            break

    return runs
```

I sort the list by repeatedly comparing two neighbouring elements and swapping them if they are in the wrong order. Each pass pushes the largest unsorted value to the end.

I also use a swapped flag to stop early. If I go through the whole list without swapping anything, it means the list is already sorted, so I exit immediately instead of continuing unnecessary passes.

Step-by-Step Trace:

Starting Position	Score	Time Left	Status
runs[0]	1500	95 s	LOSE
runs[1]	3200	187 s	WIN
runs[2]	3200	210 s	WIN

Pass 1:

I compare neighbouring runs and swap them if they are in the wrong order (lower score or worse time first). After going through the list, the largest run ends up at the correct position on the right.

Pass 2:

I compare again, but this time no swaps are needed because the list is already sorted.

Since no swaps happen in Pass 2, I stop immediately (early exit), because the list is already in the correct order.

Final result:

Final Rank	Run	Score	Time	Reason
1st	Run 1 (life 2)	3200	187 s	Same score as Run 2 but lower time = more efficient
2nd	Run 2 (life 3)	3200	210 s	Same score, more time left = slightly easier run
3rd	Run 3 (life 1)	1500	95 s	Lowest score

7. Pygame Technical Features

7.1 Custom Physics Engine

I store all physics values as named constants in `const.py`. This makes it easy for me to adjust the game's behaviour in one place. For example, if I change `GRAVITY`, it automatically affects everything that depends on it — including the player's jump, enemy falling, fireball movement, and particle effects — without having to search through the whole code to update multiple values.

Constant	Value	Effect
<code>GRAVITY</code>	0.09	Added to <code>y_vel</code> every frame when airborne — small value creates the floaty Mario feel
<code>JUMP_POWER</code>	5.5	The initial upward <code>y_vel</code> applied when the jump button is pressed
<code>FALL_MULTIPLIER</code>	2.0	Multiplies gravity while falling — makes descent faster than ascent
<code>LOW_JUMP_MULTIPLIER</code>	3.0	Multiplies gravity while rising with button released — creates short hops
<code>MAX_FALL_SPEED</code>	5.5	Caps downward <code>y_vel</code> — prevents Mario from falling through thin platforms
<code>MAX_MOVE_SPEED</code>	2.0	Maximum horizontal speed during normal walking
<code>MAX_FASTMOVE_SPEED</code>	3.0	Maximum horizontal speed while Shift (run) is held

7.2 Spatial Hash Collision Detection

A naive collision system would check every entity against every tile on the map, which would create thousands of checks every frame and slow the game down. Instead, I use the 2D `self.map` array as a spatial lookup.

From an entity's pixel position, I convert it into tile coordinates by dividing by 32. Then I only check the 14 tiles around that position instead of the whole map. This keeps collision detection fast and consistent, because the number of checks does not increase even if the map becomes larger.

```
def get_blocks_for_collision(self, x, y):
    # Returns a tuple of blocks around the given (x, y) coordinates to check for collisions.
    # This is used to determine if the player or mobs are colliding with the environment.
    return (
        self.map[x][y - 1],
        self.map[x][y + 1],
        self.map[x][y],
        self.map[x - 1][y],
        self.map[x + 1][y],
        self.map[x + 2][y],
        self.map[x + 1][y - 1],
        self.map[x + 1][y + 1],
        self.map[x][y + 2],
        self.map[x + 1][y + 2],
        self.map[x - 1][y + 1],
        self.map[x + 2][y + 1],
        self.map[x][y + 3],
        self.map[x + 1][y + 3]
    )
```

7.3 Camera System

camera.py handles scrolling the map as Mario moves. It calculates a horizontal offset so Mario stays centred on the screen, then clamps that value so the camera never goes beyond the edges of the map. The camera does not move up or down — it only follows Mario left and right.

```
def complex_camera(self, target_rect):
    # Calculate X and Y to keep Mario in the center of the screen
    x, y = target_rect.x, target_rect.y
    width, height = self.rect.width, self.rect.height
    x, y = (-x + WINDOW_W / 2 - target_rect.width / 2), (-y + WINDOW_H / 2 - target_rect.height)

    # Stop the camera from scrolling past the map edges
    x = min(0, x)
    x = max(-(self.rect.width - WINDOW_W), x)
    y = WINDOW_H - self.rect.h

    return pg.Rect(x, y, width, height)

def apply(self, target):
    # Adjust an object's position based on camera scroll
    return target.rect.x + self.rect.x, target.rect.y

def update(self, target):
    # Move the camera to track the target (Mario)
    self.rect = self.complex_camera(target)

def reset(self):
    # Reset camera to the start of the level
    self.rect = pg.Rect(0, 0, self.rect.w, self.rect.h)
```

The important idea is that the camera does not actually move any game objects. It only changes how they are displayed on the screen. All objects keep their real world positions unchanged, and the `apply()` method just adds the camera's x offset to convert them into screen positions. Because of this, the physics system can work independently and does not need to know anything about the camera.

7.4 Sprite Animation System

Mario has 50 sprite frames in total, covering small, big, and fire power states, including left/right directions and special transition and death frames. I store all of these in a single flat list when the game starts.

Instead of using many if statements to choose the correct frame, I calculate the index directly using a formula. This makes sprite selection faster, simpler, and easier to manage.

```
def set_image(self, image_id):  
    # "Dead" sprite  
    if image_id == len(self.sprites):  
        self.image = self.sprites[-1]  
  
    elif self.direction:  
        self.image = self.sprites[image_id + self.powerLVL * 8]  
    else:  
        self.image = self.sprites[image_id + self.powerLVL * 8 + 24]
```

```
# 0 Small, stay  
pg.image.load('images/Mario/mario.png'),  
  
# 1 Small, move 0  
pg.image.load('images/Mario/mario_move0.png'),  
  
# 2 Small, move 1  
pg.image.load('images/Mario/mario_move1.png'),  
  
# 3 Small, move 2  
pg.image.load('images/Mario/mario_move2.png'),  
  
# 4 Small, jump  
pg.image.load('images/Mario/mario_jump.png'),  
  
# 5 Small, end 0  
pg.image.load('images/Mario/mario_end.png'),  
  
# 6 Small, end 1  
pg.image.load('images/Mario/mario_end1.png'),  
  
# 7 Small, stop  
pg.image.load('images/Mario/mario_st.png'),  
  
# 8 Big, stay  
pg.image.load('images/Mario/mario1.png'),  
  
# 9 Big, move 0  
pg.image.load('images/Mario/mario1_move0.png'),  
  
# 10 Big, move 1  
pg.image.load('images/Mario/mario1_move1.png'),  
  
# 11 Big, move 2  
pg.image.load('images/Mario/mario1_move2.png'),  
  
# 12 Big, jump
```

The $\text{powerLVL} * 8$ part is what makes this approach clean. Instead of writing separate if conditions for small, big, and fire Mario, I use this calculation to jump directly to the correct section of the sprite list.

This also makes the system easy to extend. If I add another power level in the future, I only need to add 8 new frames and update `load_sprites()`. I would not need to change `set_image()` at all.

8. Reflections and Improvements

8.1 What Worked Well

_The 26-module structure worked really well for me because I could add the leaderboard feature at the end without changing any existing game or physics code. This would have been much harder in a single-file design.

_Passing the core object as a parameter helped me avoid circular imports completely, since no modules had to import each other directly. This kept the project cleaner and easier to manage.

_Putting all physics values in `const.py` made tuning very simple. For example, adjusting `JUMP_POWER` or `GRAVITY` immediately affected the whole game consistently.

_The dual `pos_x / rect.x` approach fixed a common velocity accumulation issue in Pygame and made movement smoother and more stable.

_Using an Entity base class reduced a lot of duplicated code, since Goombas, Koopas, and Mushrooms only needed to define their own behaviors like collision and death logic.

8.2 Limitations

_My `map.py` has grown to around 350 lines, so I think the world loading part (`loadWorld_11`) should be moved into a separate `WorldLoader` class. This would make the `Map` class cleaner and follow the single responsibility principle better.

_The leaderboard currently resets every time I restart the game. I could improve this by saving the data to a JSON file using Python's `json` module so scores can persist between sessions.

_The game currently only supports World 1-1. I think adding a more general level loading system that reads level names from a config file would make it much easier to expand the game with new worlds.

_The Koopa shell collision check (`check_collision_with_mobs`) loops through all enemies every frame, which is fine for now. However, if the game scales up, I would need a more efficient method like spatial partitioning to improve performance.

8.3 Self-Assessment

Based on my work in this project I believe the project meets Distinction-level requirements and demonstrates several aspects of High Distinction performance. The custom Bubble Sort with a composite comparison key and early-exit optimization meets the algorithm requirement. The full Pygame game with custom physics, multiple enemy behaviors, and a four-state game system meets the technical requirements.

I also implemented several design patterns and reflected on them properly, which shows I went beyond just meeting the minimum requirements. Overall, I made sure to explain my design decisions clearly so that they could be understood in an interview or technical discussion.

References / Learning Resources:

DaFluffyPotato. (n.d.). *Pygame platformer tutorial - full course* [Video]. YouTube. <https://www.youtube.com/watch?v=2gABYM5M0ww>

Easy Learning. (n.d.). *Mario game development in Python* [Video playlist]. YouTube. <https://www.youtube.com/playlist?list=PL78sHffDjI74o8FSaoBRa0PjV1i81-cH6>

Clear Code. (n.d.). *Master Python by making 5 games – the new ultimate introduction to Pygame* [Video]. YouTube.

<https://www.youtube.com/watch?v=8OMghdHP-zs>